

# **Introduction 2**

# Topics

- **Objects**
  - types of objects
  - creating/naming an object
  - data types
- **Operators**
  - mathematical
  - relational
  - logical

# Objects

# Objects

Everything we can create in R is called an object (e.g., numbers, vectors, matrices, functions).

```
1 number = 4
2 number
```

```
[1] 4
```

```
1 my_vector = c(1,2,3,4)
2 my_vector
```

```
[1] 1 2 3 4
```

```
1 my_matrix = matrix(nrow = 2, ncol = 2, data = my_vector)
2 my_matrix
```

```
      [,1] [,2]
[1,]    1    3
[2,]    2    4
```

# Functions

Everything we do in R is calling **functions** on objects.

Functions allow us to **create** and **modify** objects.

```
1 my_vector
```

```
[1] 1 2 3 4
```

```
1 mean(x = my_vector)
```

```
[1] 2.5
```

```
1 number
```

```
[1] 4
```

```
1 number = mean(my_vector)
2 number
```

```
[1] 2.5
```

# Creating/naming objects

Objects can be created with either the `<-` or the `=` command

```
1 x1 = 3 # name = object
2 x1
```

```
[1] 3
```

```
1 x2 <- 3 # name <- object
2 x2
```

```
[1] 3
```

```
1 x1 == x2 # are the two objects identical?
```

```
[1] TRUE
```

You can choose the command you prefer; the important thing is to be consistent.

# Rules for naming objects

- A name must start with a letter and can contain letters, numbers, underscores ( \_ ), or dots (.).
- It may also start with a dot (.), but in that case it cannot be followed by a number.

```
1 .3 = 3
```

```
Error in `0.3 = 3`:  
! invalid (do_set) left-hand side to assignment
```

```
1 .x = 3
```

- It must not contain special characters such as #, &, \$, ?, etc.
- It must not be a reserved word, i.e. one of the words used by R with a special meaning (?reserved).

```
1 function = 3
```

```
Error in parse(text = input): <text>:1:10: unexpected '='  
1: function =  
      ^
```

```
1 TRUE = 3
```

```
Error in `TRUE = 3`:  
! invalid (do_set) left-hand side to assignment
```

```
1 if = 3
```

```
Error in parse(text = input): <text>:1:4: unexpected '='  
1: if =  
    ^
```

# Conventions

Some names are not forbidden but are discouraged

```
1 T
```

```
[1] TRUE
```

```
1 TRUE
```

```
[1] TRUE
```

```
1 T == TRUE
```

```
[1] TRUE
```

```
1 T = 1; T
```

```
[1] 1
```

```
1 T == TRUE
```

```
[1] TRUE
```

```
1 sum(2,3)
```

```
[1] 5
```

```
1 sum = 4; sum
```

```
[1] 4
```

Using “.” in object names (for example, “my.data”) is fine in **R** but is not allowed in **Python**, where “.” is part of the language syntax.

Across languages, the *naming conventions* for longer, multi-word variable names favour **snake\_case** (for example, “**my\_data**”) or **camelCase** (for example, “**myData**”), and **abbreviations** where appropriate (for example, “**unipdData**” rather than “university\_of\_padova\_dataset”).

# R is case-sensitive

```
1 Name = "Margherita"  
2  
3 name = "margherita"  
4  
5 Name
```

```
[1] "Margherita"
```

```
1 name
```

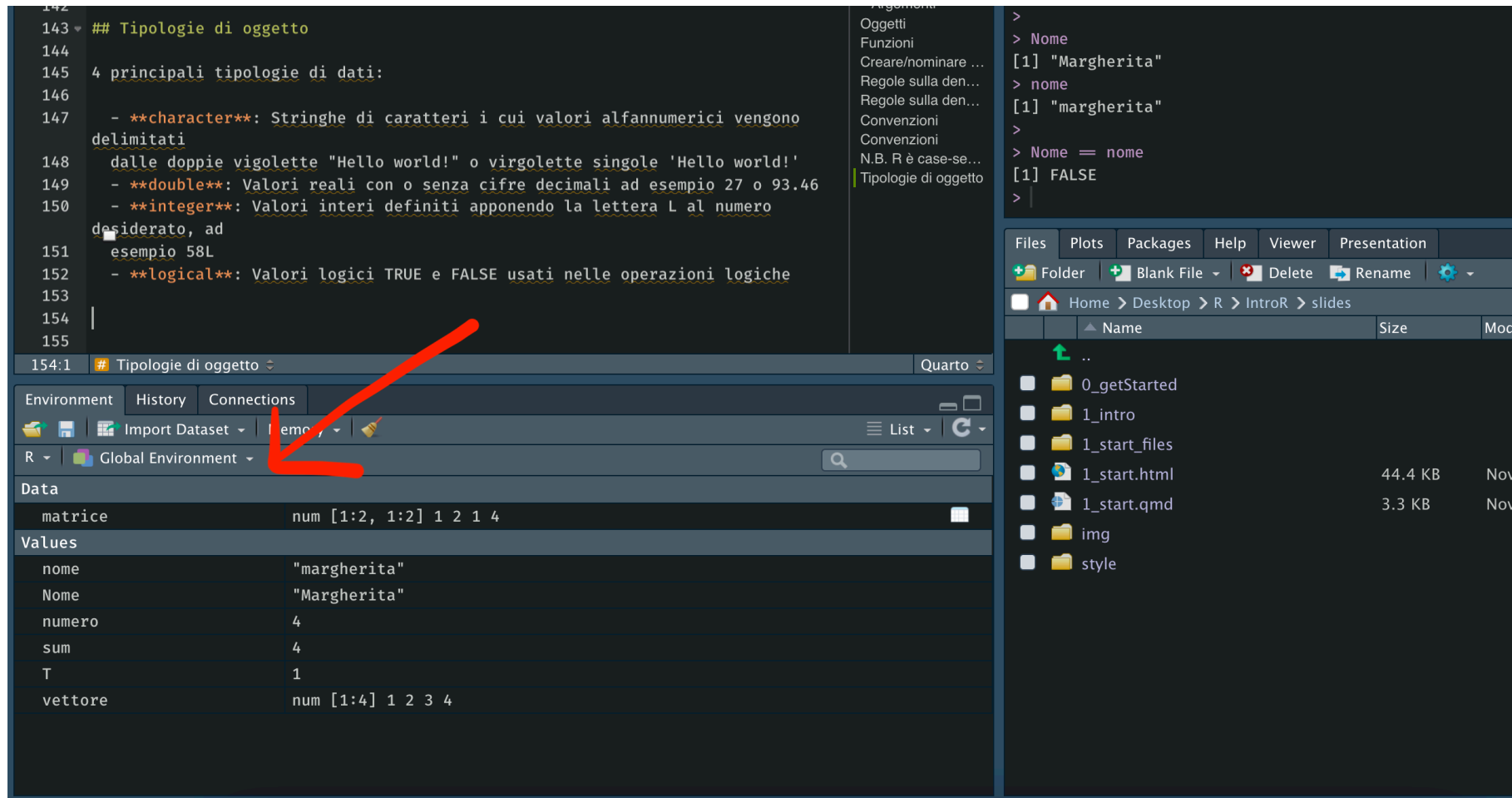
```
[1] "margherita"
```

```
1 Name == name
```

```
[1] FALSE
```

# Where are objects stored?

By default, objects are created in the **global environment**, which can be accessed with `ls()` or viewed in R Studio together with some additional information:



The screenshot displays the R Studio interface. The top-left pane shows R code with comments in Italian describing object types. The top-right pane shows the output of a command, listing object names and their values. The bottom-left pane shows the 'Global Environment' with a list of objects and their values. A red arrow points to the 'Global Environment' tab. The bottom-right pane shows a file explorer view of the 'slides' directory.

```
142
143 ## Tipologie di oggetto
144
145 4 principali tipologie di dati:
146
147 - **character**: Stringhe di caratteri i cui valori alfanumerici vengono
delimitati
148 dalle doppie virgolette "Hello world!" o virgolette singole 'Hello world!'
149 - **double**: Valori reali con o senza cifre decimali ad esempio 27 o 93.46
150 - **integer**: Valori interi definiti apponendo la lettera L al numero
desiderato, ad
151 esempio 58L
152 - **logical**: Valori logici TRUE e FALSE usati nelle operazioni logiche
153
154
155
```

```
>
> Nome
[1] "Margherita"
> nome
[1] "margherita"
>
> Nome == nome
[1] FALSE
>
```

Environment History Connections

R Global Environment

| Data    |                        |
|---------|------------------------|
| matrice | num [1:2, 1:2] 1 2 1 4 |

| Values  |                   |
|---------|-------------------|
| nome    | "margherita"      |
| Nome    | "Margherita"      |
| numero  | 4                 |
| sum     | 4                 |
| T       | 1                 |
| vettore | num [1:4] 1 2 3 4 |

Files Plots Packages Help Viewer Presentation

Folder Blank File Delete Rename

Home Desktop R IntroR slides

| Name          | Size    | Mod |
|---------------|---------|-----|
| ..            |         |     |
| 0_getStarted  |         |     |
| 1_intro       |         |     |
| 1_start_files |         |     |
| 1_start.html  | 44.4 KB | Nov |
| 1_start.qmd   | 3.3 KB  | Nov |
| img           |         |     |
| style         |         |     |

# Removing objects

We can remove an object from our environment with the `rm("objectname")` command.

We can also completely clear/empty our environment with the `rm(list = ls())` command.

# Data types

# Data types

- **character**: Character strings whose alphanumeric values are delimited by double quotes “Hello world!” or single quotes ‘Hello world!’
- **numbers**:
  - **double**: Real values with or without decimal digits, for example 27 or 93.46
  - **integer**: Integer values defined by appending the letter L to the number, for example 58L

# Mathematical Operators

| Function | What does it do? | Example   | Result |
|----------|------------------|-----------|--------|
| +        | addition         | 5.4 + 6.1 | 11.5   |
| -        | subtraction      | 9 - 4.3   | 4.7    |
| *        | multiplication   | 7 * 1.4   | 9.8    |
| /        | division         | 9/3       | 3      |
| %%       | remainder        | 9%%2      | 1      |
| ^        | power            | 15 ^ 2    | 225    |

| Function           | What does it do?     | Example                      | Result |
|--------------------|----------------------|------------------------------|--------|
| <code>abs</code>   | absolute value       | <code>abs(-8)</code>         | 8      |
| <code>sqrt</code>  | square root          | <code>sqrt(225)</code>       | 15     |
| <code>exp</code>   | exponential function | <code>exp(0)</code>          | 1      |
| <code>log</code>   | natural logarithm    | <code>log(1)</code>          | 0      |
| <code>round</code> | rounding, integer    | <code>round(1.738)</code>    | 2      |
| <code>round</code> | rounding             | <code>round(1.738, 2)</code> | 1.74   |

# Mathematical Operations

The order of operations in R follows the rules of mathematics.

## Examples

```
1 # Without parentheses
2 1 + 2 * 3
```

```
[1] 7
```

```
1 # With parentheses
2 (1 + 2) * 3
```

```
[1] 9
```

# Relational Operators

In R we can evaluate whether a given relation is true or false. R will evaluate the statement and return **TRUE** if the statement is true or **FALSE** if the statement is false.

| Function                | Name             | Example                      | Result |
|-------------------------|------------------|------------------------------|--------|
| <code>==</code>         | equal            | <code>30 == 30</code>        | TRUE   |
| <code>!=</code>         | not equal        | <code>30 != 30</code>        | FALSE  |
| <code>&gt;/&gt;=</code> | greater/or equal | <code>30 &gt; 10</code>      | TRUE   |
|                         |                  | <code>30 &gt;= 10</code>     | TRUE   |
| <code>&lt;/&lt;=</code> | less/or equal    | <code>30 &lt; 10</code>      | FALSE  |
|                         |                  | <code>10 &lt;= 10</code>     | TRUE   |
| <code>%in%</code>       | inclusion        | <code>10%in%c(1,2,10)</code> | TRUE   |

## It doesn't only work with numbers!

```
1 Name = "Margherita"  
2  
3 name = "margherita"  
4  
5 Name == name
```

```
[1] FALSE
```

PS. Remember that `=` is different from `==`

# Logical Operators

In R we can combine several relations to evaluate a given statement.

```
1 x = 30 # Assign the value 30 to x.
```

| Function | Name                       | Example     | Result |
|----------|----------------------------|-------------|--------|
| &        | Conjunction (AND)          | x>25 & x<60 | TRUE   |
|          | Inclusive disjunction (OR) | x>25   x>60 | TRUE   |
| !        | Negation (NOT)             | !(x<18)     | TRUE   |

# Logical Operators

```
1 x = 30 # Assign the value 30 to x.  
2 x
```

[1] 30

```
1 x>25 & x>60
```

[1] FALSE

```
1 x>25 | x>60
```

[1] TRUE

```
1 !(x>18)
```

[1] FALSE

# TRUE is equivalent to 1 and FALSE to 0

```
1 TRUE == 1
```

```
[1] TRUE
```

```
1 TRUE == 2
```

```
[1] FALSE
```

```
1 FALSE == 0
```

```
[1] TRUE
```

```
1 FALSE == 1
```

```
[1] FALSE
```

I create an example vector containing the words home (repeated 4 times) and shop (repeated 2 times):

```
1 example = c(rep("home", times = 4), rep("shop", times = 2)) #?rep
2 example
```

```
[1] "home" "home" "home" "home" "shop" "shop"
```

```
1 # How many words in example are equal to home?
2 example == "home"
```

```
[1] TRUE TRUE TRUE TRUE FALSE FALSE
```

```
1 sum(example == "home") # why?
```

```
[1] 4
```

# Order of Evaluation

When evaluating whether statements are true, R performs the operations in the following order:

1. Mathematical operators (e.g.,  $\wedge$ ,  $*$ ,  $/$ ,  $+$ ,  $-$ , etc.)
2. Relational operators (e.g.,  $<$ ,  $>$ ,  $<=$ ,  $>=$ ,  $==$ ,  $!=$ )
3. Logical operators (e.g.,  $!$ ,  $\&$ ,  $|$ )

If in doubt about the order in which operations are executed, the best thing to do is to use round brackets ( ) to remove any possible ambiguity.

# Let's practice!

Open this link and keep it open: <https://etherpad.wikimedia.org/p/arca-corsoR>

